

SMART METRO TICKET BOOKING SYSTEM

A ServiceNow-Based Application for Metro Ticket Booking,
Passenger Management, Fare Calculation and Ticket Generation

Final Project Report

Project Name	Smart Metro Ticket Booking System
Domain	ServiceNow Application Development
Document Type	Consolidated Project Report
Team ID	PNT2025TMID-XXXXXX

Table of Contents

1. Introduction	4
1.1 Project Overview.....	4
1.2 Purpose.....	4
2. Ideation Phase	5
2.1 Problem Statement.....	5
2.2 Empathy Map Canvas	5
2.3 Brainstorming.....	6
3. Requirement Analysis	8
3.1 Customer Journey Map	8
3.2 Solution Requirement	9
3.3 Data Flow Diagram.....	10
3.4 Technology Stack.....	12
4. Project Design	14
4.1 Problem Solution Fit	14
4.2 Proposed Solution	14
4.3 Solution Architecture.....	15
5. Project Planning & Scheduling	18
5.1 Project Planning.....	18
6. Functional and Performance Testing	21
6.1 Functional Testing (UAT).....	21
6.2 Performance Testing	22
7. Results	24
7.1 Output Screenshots	24
8. Advantages & Disadvantages	34
9. Conclusion	35
10. Future Scope	36

11. Appendix	37
Source Code	37
GitHub Link	38
Project Demo Link	38

1. Introduction

1.1 Project Overview

The Smart Metro Ticket Booking System (SMTGS) is a ServiceNow-based application that digitises the end-to-end metro ticketing journey — from ticket booking and passenger management to fare calculation and ticket generation. The system replaces the traditional counter-driven, paper-based ticketing model with a single, automated platform accessible through the ServiceNow Service Portal, a companion mobile application, and WhatsApp, allowing commuters to plan, purchase, pay for and validate their journeys without visiting a staffed counter.

Built on the ServiceNow platform, the solution combines a Service-Catalog-driven booking experience with automated fare-logic scripting, integrated digital payments (UPI, cards and wallets), QR-code ticket issuance, and Automatic Fare Collection (AFC) gate validation. Role-based dashboards give Station Managers, the Metro Operations Team and IT Admins real-time visibility into bookings, revenue and station-level activity, closing the operational blind spots created by manual processes.

The project follows a structured, phase-wise delivery approach — Ideation, Requirement Analysis, Project Design, Project Planning, Development and Testing — with every design decision traced back to an evidenced commuter or operator pain point, as documented across the phases assembled in this report.

1.2 Purpose

The purpose of this document is to consolidate every phase of the Smart Metro Ticket Booking System project — ideation, requirement analysis, design, planning, and functional/performance testing — into a single, coherent project report. It is intended to:

- Provide a single reference that traces the project from problem identification through to a working, tested solution.
- Demonstrate how each design and engineering decision is grounded in a real, evidence-backed commuter or operator need rather than an assumption.
- Give evaluators, stakeholders and future contributors a clear record of scope, architecture, planning artifacts, test approach and results.
- Serve as the definitive submission document summarising the outcome of the ideathon-to-delivery project lifecycle for the Smart Metro Ticket Booking System.

2. Ideation Phase

The Ideation Phase establishes the human-centred foundation of the project: understanding who the system is for, generating a broad pool of ideas without judgement, and converging those ideas into a single, evidence-backed problem statement before any solution design begins.

2.1 Problem Statement

Commuters using the metro today face a fragmented, manual ticketing experience. Long queues at counters, error-prone manual fare calculation, absence of a centralised passenger record, and lack of automation increase wait times and operational overhead for metro staff. There is no unified platform that ties booking, passenger data, fare logic and ticket issuance together in one automated, auditable workflow.

How might metro operations replace slow, manual, error-prone ticketing with a single automated ServiceNow-based system that handles booking, passenger management, fare calculation and ticket generation end-to-end — reducing wait time, human error and administrative load?

Observed Pain Points

Pain Point	Impact	Affected Stakeholder
Long queues at manual counters	Increased boarding time, commuter frustration	Passenger
Manual fare calculation errors	Revenue leakage, disputes	Operator / Passenger
No centralised passenger records	Difficult to track passes, history, refunds	Operator
Paper ticket dependency	Higher printing cost, environmental impact	Operator
No real-time visibility for staff	Slow issue resolution, poor reporting	Admin / Staff
Inconsistent refund handling	Commuter distrust, repeated support requests	Passenger / Support

Root Cause (5 Whys)

Question	Answer
Why do commuters face long queues?	Ticketing still relies on manual counters for most passengers
Why does it rely on manual counters?	No self-service digital booking option is available end-to-end
Why is there no digital option?	Booking, fare and ticketing were never unified onto one platform
Why were they never unified?	Each function was historically built or operated as a separate manual step
Why does that matter (root cause)?	Without one automated system, every stage stays manual, slow and error-prone

2.2 Empathy Map Canvas

Empathy mapping is the first step of the design-thinking process. Before a solution can be designed, the team must deeply understand the people who will use it. For the Smart Metro Ticket Booking System, this means stepping into the shoes of a daily metro commuter — observing their environment, listening to their conversations, and understanding what they think, feel, say and do while trying to book and use a metro

ticket.

The map was built by combining on-ground observation at ticket counters and gates, short informal interviews with commuters of different age groups and travel frequency, and a review of commonly reported complaints about queues, recharge and payment failures, focused on peak-hour touch-points where friction is most visible.

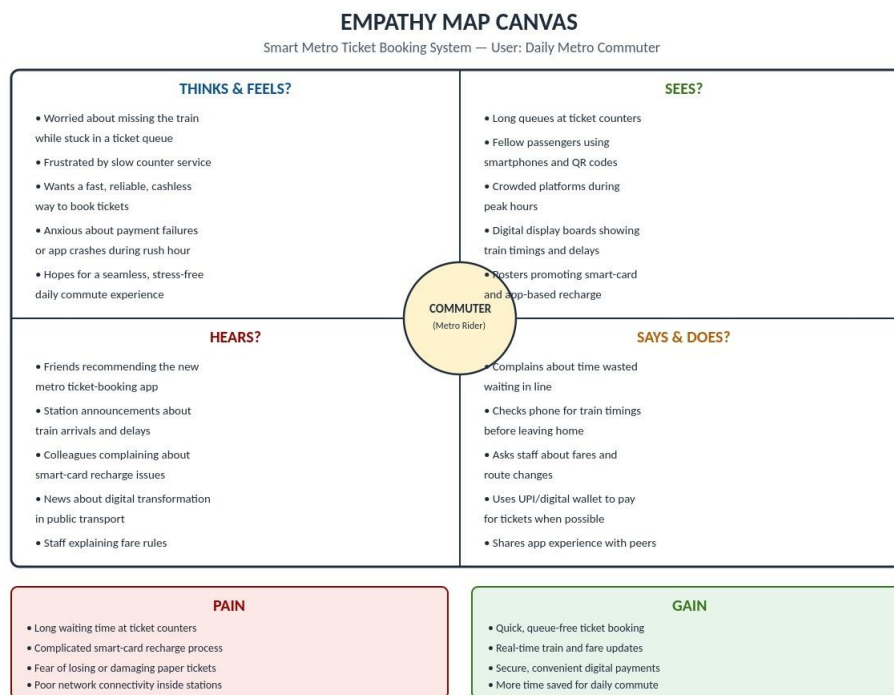


Figure 2.1 — Empathy Map Canvas for the daily metro commuter persona

Pain Points & Gains Summary

Pain	Gain
Long waiting time at ticket counters during peak hours	Quick, queue-free ticket booking experience
Complicated, multi-step smart-card recharge process	Real-time train and fare updates on the go
Fear of losing or damaging paper tickets	Secure, convenient digital payment options
Poor network connectivity inside underground stations	More time saved for the daily commute

Key takeaways: commuters value speed and predictability above all else; emotional friction is often triggered by process complexity rather than wait time alone; digital literacy varies widely, so the solution must stay simple for first-time app users; and real-time information reduces uncertainty and improves the overall commuting experience.

2.3 Brainstorming

The brainstorming session generated a wide, judgement-free pool of ideas to modernise metro ticketing through automation on ServiceNow, spanning booking, passenger management, fare calculation and ticket generation. The session followed three guiding principles: defer judgement, encourage wild ideas, and go for quantity — with feasibility and prioritisation deferred to a later phase.

Idea	Description	Category
Self-service Booking Portal	Web/mobile portal on ServiceNow Service Portal for instant ticket booking	Booking
Auto Fare Engine	Rule-based fare calculation using distance/zone tables	Fare
Digital Passenger Profile	Central record storing passenger history, passes, preferences	Passenger Mgmt
QR / Digital Ticket Generation	Auto-generate scannable QR ticket as a record on confirmation	Ticketing
Smart Notifications	SMS/Email/Push alerts for booking confirmation, reminders, refunds	Engagement
Admin Dashboard & Reports	Analytics dashboards for ridership, revenue, peak-hour load	Analytics
Chatbot Assistance	Virtual Agent for FAQs, booking help, complaint logging	Support
Group / Family Booking	Book multiple tickets under one transaction with shared fare split	Booking
Auto Refund Engine	Automatic refund trigger for cancelled or delayed journeys	Fare
Contactless Gate Validation	QR ticket scanned directly at entry/exit gates, no physical check	Ticketing
Frequent Traveler Pass	Auto-suggest monthly/weekly pass based on travel pattern	Passenger Mgmt
Multilingual Interface	Booking portal available in regional languages for wider accessibility	Engagement
Peak-Hour Demand Alerts	Warns commuters of crowded trains and suggests off-peak slots	Analytics
Lost Ticket Recovery	Self-service flow to reissue a digital ticket without staff involvement	Support
Accessibility Mode	Larger text, screen-reader support for elderly and differently-abled commuters	Engagement
Staff Duty Roster Sync	Links counter/gate staff scheduling with expected passenger volume	Passenger Mgmt

Idea Categorisation

Theme	Ideas Grouped Under It
Booking Experience	Self-service Booking Portal, Group/Family Booking, Multilingual Interface
Fare & Payments	Auto Fare Engine, Auto Refund Engine
Ticket Lifecycle	QR/Digital Ticket Generation, Contactless Gate Validation, Lost Ticket Recovery
Passenger Relationship	Digital Passenger Profile, Frequent Traveler Pass, Staff Duty Roster Sync
Communication & Support	Smart Notifications, Chatbot Assistance, Accessibility Mode
Operations & Insight	Admin Dashboard & Reports, Peak-Hour Demand Alerts

Booking, fare and ticketing formed the essential core that every other idea supports, with automation as the common thread across nearly every idea raised. High-value, low-effort ideas were carried forward as MVP candidates into the Requirement Analysis phase.

3. Requirement Analysis

The Requirement Analysis phase translates the converged problem statement into concrete functional and non-functional requirements, maps how the commuter actually experiences the journey end to end, models how data moves through the system, and defines the technology stack that will realise it.

3.1 Customer Journey Map

The Customer Journey Map traces the commuter's experience across every stage of a metro trip — from journey planning through booking, payment, gate entry, travel and gate exit — capturing the commuter's actions, touch-points, emotional state and pain points at each stage. It builds directly on the empathy map findings and grounds the requirements below in the commuter's real, moment-by-moment experience rather than an assumed workflow.

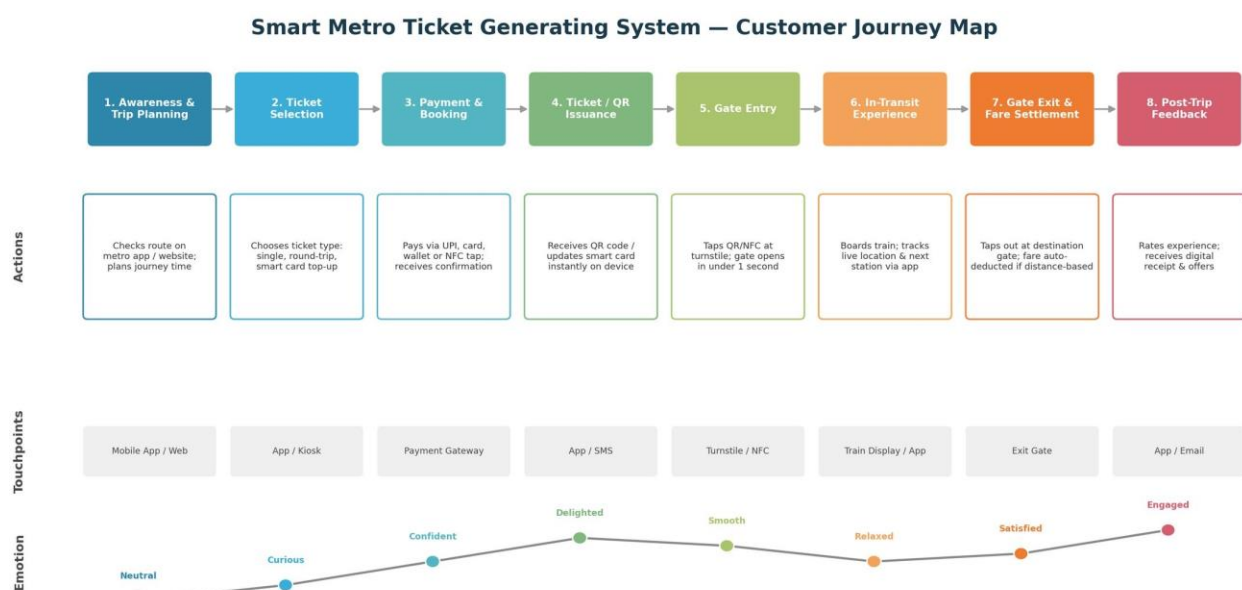


Figure 3.1 — Customer Journey Map across the pre-booking, booking, payment, travel and post-journey stages

Stage	Commuter Action	Touch-point	Pain Point Today
Plan	Checks route, timing and fare	Station signage / word of mouth	No single source of live fare or timing info
Book	Selects source, destination, passenger type	Ticket counter / TVM queue	Long wait, manual entry errors
Pay	Pays for the ticket	Cash at counter	Cash-only, slow, no digital record
Enter	Presents ticket at gate	Manual / token-based gate	Manual checks slow entry, enable fare evasion
Travel	Rides to destination	Platform / train	No real-time updates during disruption
Exit	Exits at destination station	Manual / token-based gate	No unified record of the completed trip

3.2 Solution Requirement

This section defines the functional and non-functional requirements for the Smart Metro Ticket Booking System, covering ticket purchase, fare calculation, payment processing, gate validation and administrative reporting across all metro stations served by the platform.

Stakeholders: Commuters, Metro Operations Team, Station Staff, Finance / Audit Team, Third-party Payment Providers, IT / Operations.

Functional Requirements

ID	Requirement	Priority
FR-01	Commuter can purchase single, round-trip and multi-ride tickets via app, web or kiosk	Must
FR-02	System calculates distance-based or zone-based fare in real time	Must
FR-03	Commuter can pay via UPI, debit/credit card, NFC wallet or smart-card balance	Must
FR-04	System issues a QR code / updates NFC smart card within 3 seconds of payment	Must
FR-05	Turnstile validates ticket and opens gate within 1 second of tap	Must
FR-06	Commuter can recharge smart card / wallet remotely	Should
FR-07	Commuter can request refund/cancellation before journey start	Should
FR-08	Admin can configure fare rules, discounts and station data	Must
FR-09	System generates revenue and ridership reports for operators	Should
FR-10	System sends push/SMS notification on ticket issuance and low balance	Could

Smart Metro Ticket Generating System — Requirements Breakdown

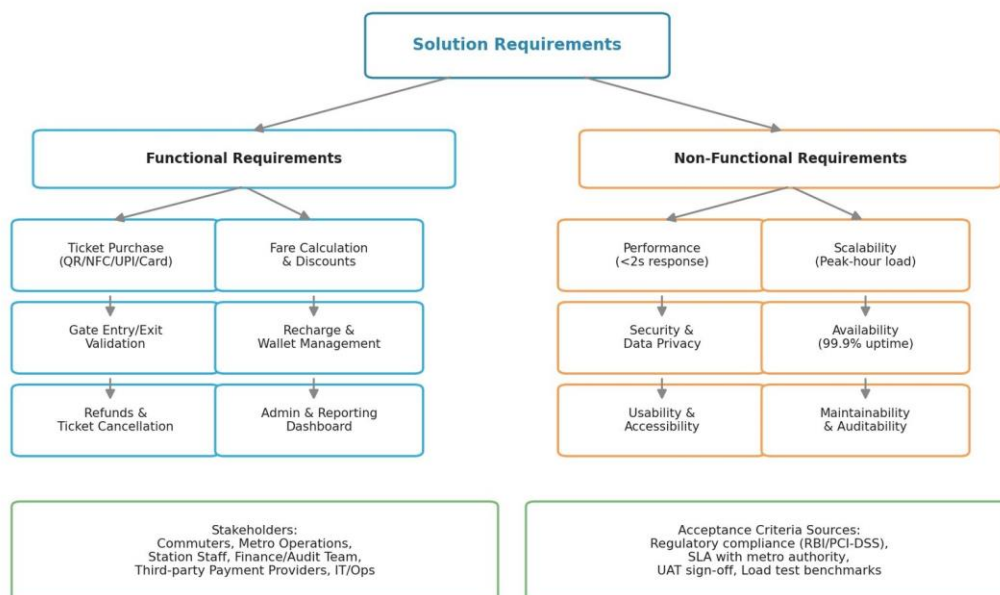


Figure 3.2 — Requirements breakdown structure (Functional / Non-Functional)

Non-Functional Requirements

- **Performance:** 95% of ticket-purchase API calls complete in under 2 seconds; gate validation under 1 second.
- **Scalability:** sustain at least 10,000 concurrent transactions during peak windows without degradation.

- **Security:** payment data encrypted in transit (TLS 1.3) and at rest; PCI-DSS and applicable data-protection compliance.
- **Availability:** 99.9% uptime for ticketing and gate-validation services, with graceful offline gate degradation.
- **Usability:** WCAG 2.1 AA accessibility, usable in at least two regional languages plus English.
- **Maintainability:** independently deployable services with full audit trails for configuration changes.

Assumptions & Constraints: commuters mostly access the system via smartphone or NFC card (cash supported at kiosks); the system must integrate with existing turnstile hardware at legacy stations; all financial transactions comply with RBI/PCI-DSS guidelines; and initial rollout is limited to a defined set of pilot stations before network-wide expansion.

3.3 Data Flow Diagram

The Data Flow Diagrams model how information moves through the system, presented at two levels: Level 0 (context diagram), showing the system as a single process bounded by its external actors, and Level 1, decomposing that process into its major internal sub-processes and data stores.

Smart Metro Ticket Generating System — DFD Level 0 (Context Diagram)

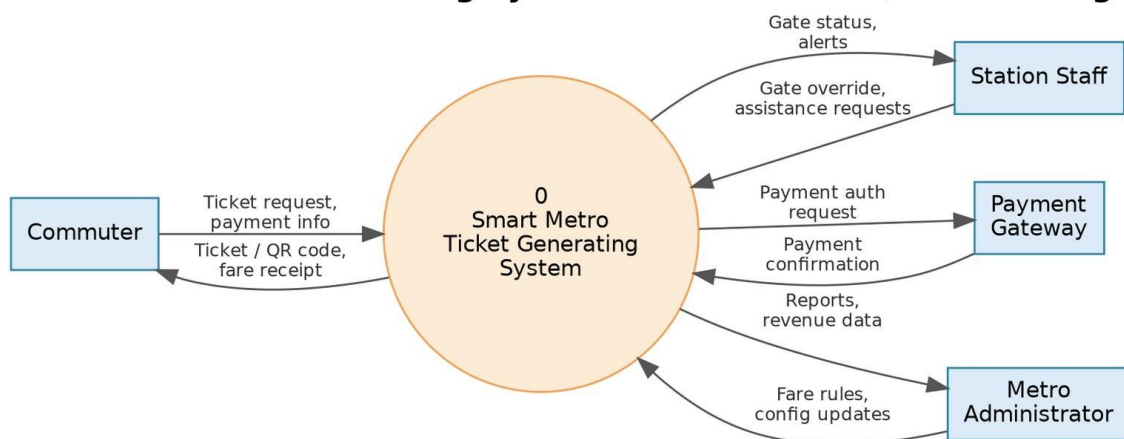


Figure 3.3a — DFD Level 0: Context diagram

Smart Metro Ticket Generating System — DFD Level 1 (Detailed)

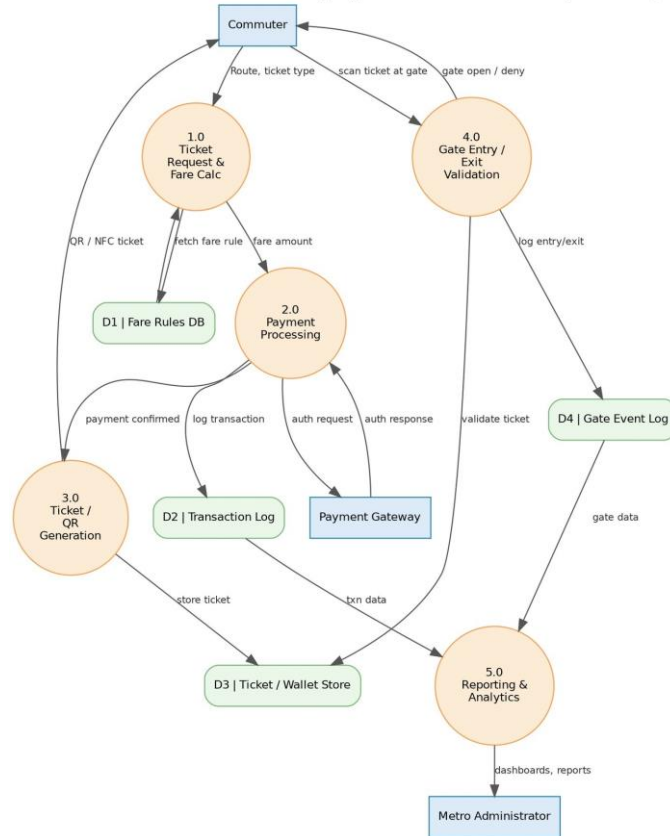


Figure 3.3b — DFD Level 1: Detailed process decomposition

Process	Description
1.0 Ticket Request & Fare Calculation	Receives route/ticket-type input and computes fare using rules cached from the Fare Rules DB
2.0 Payment Processing	Orchestrates authorization with the external Payment Gateway and writes an immutable Transaction Log entry
3.0 Ticket / QR Generation	Issues the QR code / updates the NFC smart card, persisting the ticket to the Ticket / Wallet Store
4.0 Gate Entry / Exit Validation	Validates a scanned ticket in real time and logs every tap to the Gate Event Log
5.0 Reporting & Analytics	Aggregates transaction and gate-event data into dashboards for the Metro Administrator

Representative User Stories

ID	User Story	Acceptance Criteria
US-01	As a commuter, I want to buy a single-journey ticket in the app so I can travel without visiting a counter.	Ticket/QR issued within 3s and visible offline
US-02	As a commuter, I want to see the fare before I pay so I can confirm the cost.	Fare shown prior to payment confirmation
US-03	As a commuter, I want to tap my phone at the gate so I don't need a physical ticket.	NFC/QR tap opens gate in <1s for a valid ticket
US-05	As station staff, I want to override a gate for a failed scan so commuters aren't blocked unfairly.	Override requires authentication, logged with reason code

ID	User Story	Acceptance Criteria
US-06	As a metro administrator, I want to update fare rules centrally so all stations apply them instantly.	Rule change propagates within 5 minutes, version-logged
US-08	As a commuter, I want a refund if my journey is cancelled before travel.	Refund processed within 24 hours to original payment method

3.4 Technology Stack

The Smart Metro Ticket Booking System is a distributed, cloud-native platform organised into four layers — Presentation, Application/API, Data and Infrastructure — chosen to meet the system's demanding requirements for high transaction throughput during peak commute hours, sub-second gate response times, strong payment security, and 24x7 availability across hundreds of stations.

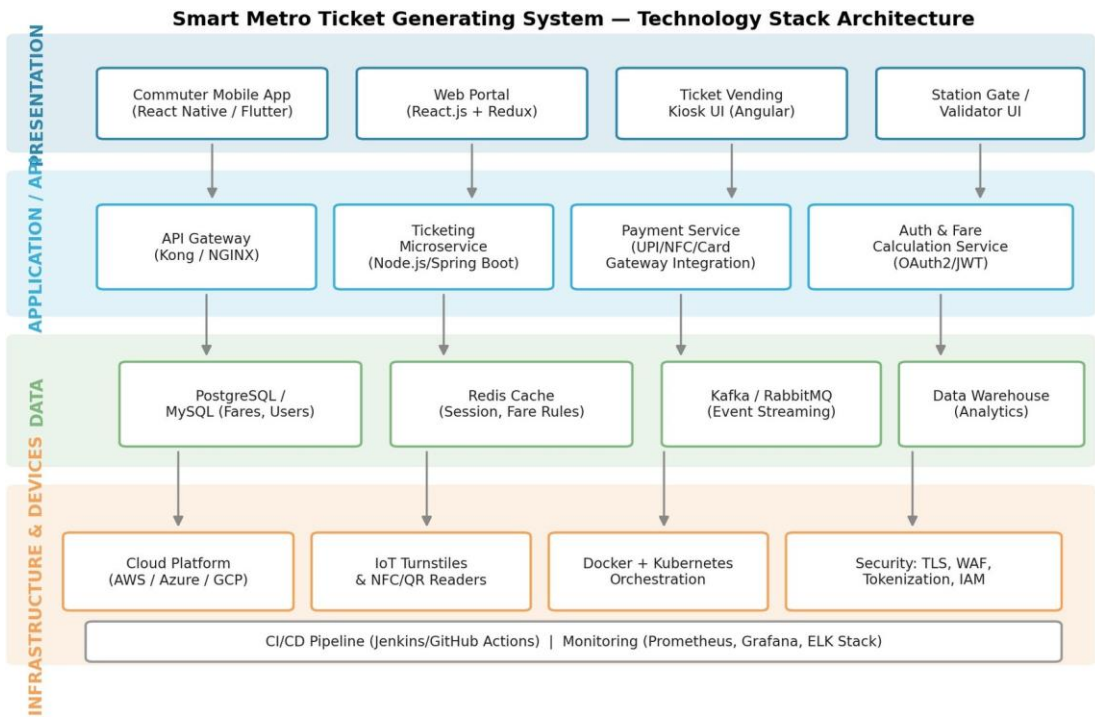


Figure 3.4 — Layered technology stack architecture

Layer	Technology / Tool	Purpose
Mobile App	Flutter / React Native	Cross-platform commuter app for iOS & Android
Web Portal	React.js, Redux, TypeScript	Self-service ticketing and account management
Kiosk / TVM UI	Angular, Electron	Touchscreen ticket vending machine interface
API Gateway	Kong / NGINX	Request routing, rate limiting, TLS termination
Ticketing Service	Node.js / Spring Boot	Core fare and ticket issuance logic
Payment Service	Java / Node.js + UPI, NFC, Card SDKs	Payment orchestration and reconciliation
Auth Service	OAuth2.0, JWT, Keycloak	Identity, session and token management
Relational DB	PostgreSQL / MySQL	Transactional data: users, fares, tickets
Cache	Redis	Low-latency fare rule and session lookups
Event Streaming	Apache Kafka / RabbitMQ	Asynchronous events between microservices
Analytics Store	Data Warehouse (Snowflake/BigQuery)	Reporting, revenue and ridership analytics

Layer	Technology / Tool	Purpose
Cloud Platform	AWS / Azure / GCP	Hosting, autoscaling, managed services
IoT Devices	NFC/QR Readers, Turnstile Controllers	Physical gate entry and exit validation
Container Platform	Docker, Kubernetes	Service packaging and orchestration
Security	TLS 1.3, WAF, IAM, Tokenization	Data protection and access control
CI/CD	Jenkins / GitHub Actions	Automated build, test and deployment pipelines
Monitoring	Prometheus, Grafana, ELK Stack	Observability, logging and alerting

4. Project Design

The Project Design phase converts validated requirements into an architecture and solution blueprint, starting with an explicit mapping from every observed problem to a specific solution capability.

4.1 Problem Solution Fit

Metro commuters today rely on a largely manual, paper-based ticketing process: tokens or paper tickets are issued at staffed counters, payment is predominantly cash, and gate checks are performed manually. Five problems, gathered from station observation and operator interviews, define the scope this solution must address.

Problem	Evidence	Solution Fit
Long queues at ticket counters	Avg. wait time ~90 sec at peak stations	Self-service app/web/kiosk purchase
Paper tickets lost/damaged	Manual reissue requests at counters	Digital QR/NFC ticket & smart wallet
Cash-heavy, slow reconciliation	~12 hrs manual daily reconciliation	Digital payments with auto reconciliation
Fare evasion at manual gates	~8% estimated fare leakage	Automated turnstile validation + audit log
No real-time operational data	Reports compiled manually, day+ lag	Real-time analytics dashboard

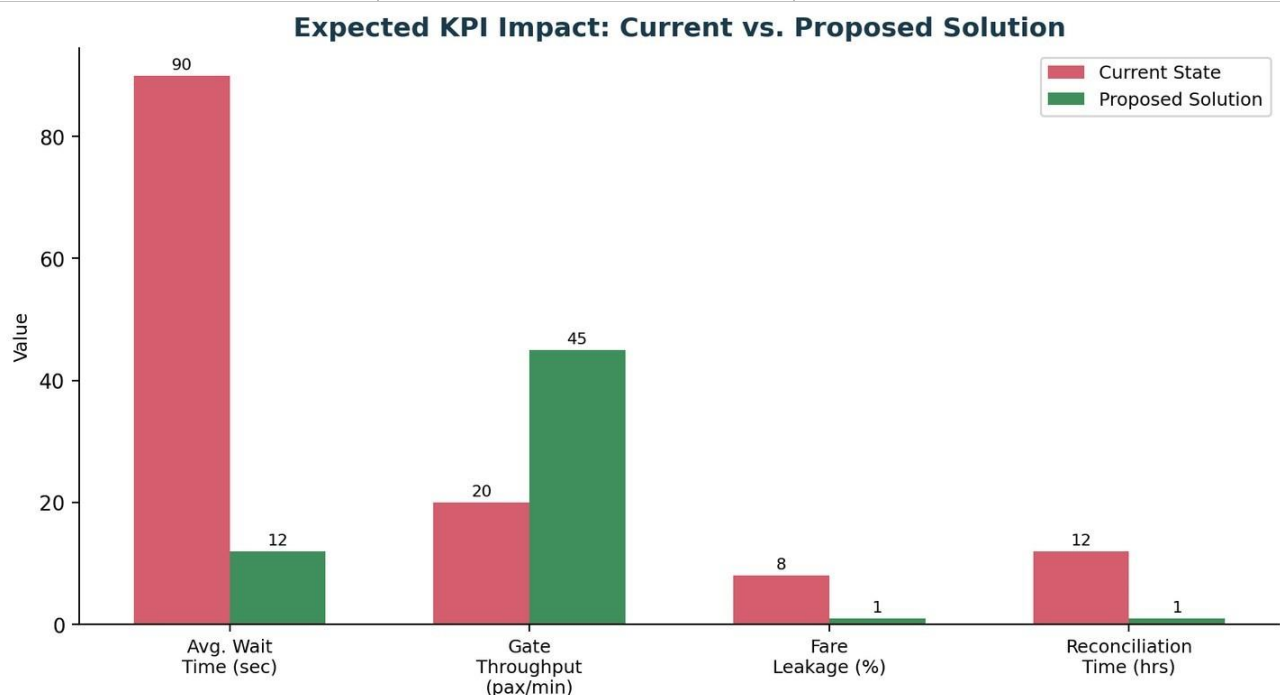


Figure 4.1 — Problem severity matrix: frequency of occurrence vs. business/commuter impact

Fare evasion and cash-heavy reconciliation sit in the highest-priority quadrant, which is why automated gate validation and digital payments were prioritised as Phase 1 pilot capabilities. Wait time, gate throughput, fare leakage and reconciliation time are the KPIs used to validate this fit once the pilot goes live.

4.2 Proposed Solution

The proposed solution is a unified, cloud-native Smart Metro Ticket Booking System that replaces manual, paper-based ticketing with a fully digital, self-service experience. Commuters purchase and validate tickets through a mobile app, web portal or station kiosk, pay via UPI, card, NFC or a unified digital wallet, and pass

through gates using contactless QR/NFC validation — with every transaction recorded in real time for operators.

Proposed Solution — Core Capabilities

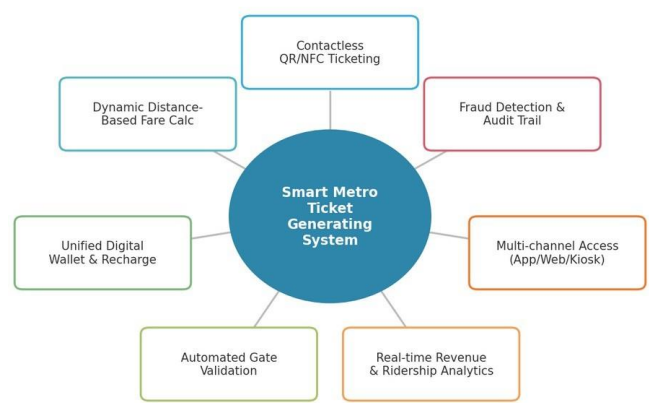


Figure 4.2 — Core capabilities radiating from the central ticketing platform

Feature	Commuter Benefit	Operator Benefit
Contactless QR/NFC ticketing	No queues; tap-and-go entry	Higher gate throughput
Dynamic distance-based fare	Pays only for distance travelled	Fairer, auditable pricing
Unified digital wallet	One balance across all channels	Reduced cash handling
Real-time analytics dashboard	Faster service issue resolution	Data-driven capacity planning
Multi-channel access	Buy via app, web or kiosk	Lower counter staffing needs

Rollout approach: Phase 1 – Pilot (4–6 stations) validates the end-to-end purchase-to-gate flow; Phase 2 – Regional rollout expands to a full line and onboards station staff; Phase 3 – Network-wide launch rolls out across all stations and hands off operations to Metro Operations and IT.

4.3 Solution Architecture

The system is built as a layered, cloud-native architecture separating concerns across five tiers — Client, Gateway, Service, Data and External Integration — following a microservices pattern where each core capability (authentication, ticketing, payment, reporting) is an independently deployable service communicating over well-defined APIs and asynchronous events.

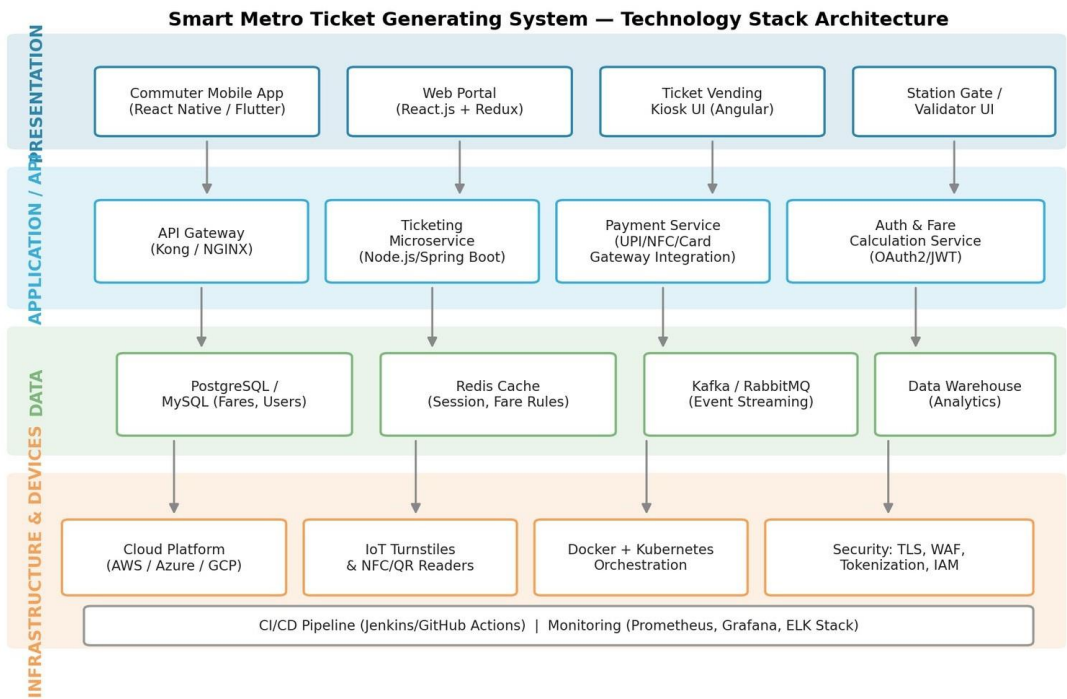


Figure 4.3a — Layered solution architecture

Tier	Components	Responsibility
Client Tier	Mobile App, Web Portal, Kiosk, Gate UI	User interaction, ticket display, offline QR cache
Gateway Tier	API Gateway / Load Balancer	Routing, TLS termination, rate limiting, auth check
Service Tier	Auth, Ticketing, Payment, Reporting microservices	Business logic, independently deployable pods
Data Tier	PostgreSQL, Redis, Kafka	Persistence, caching, event streaming
External Tier	Payment Gateway, SMS/Push Provider	Third-party integrations outside system boundary

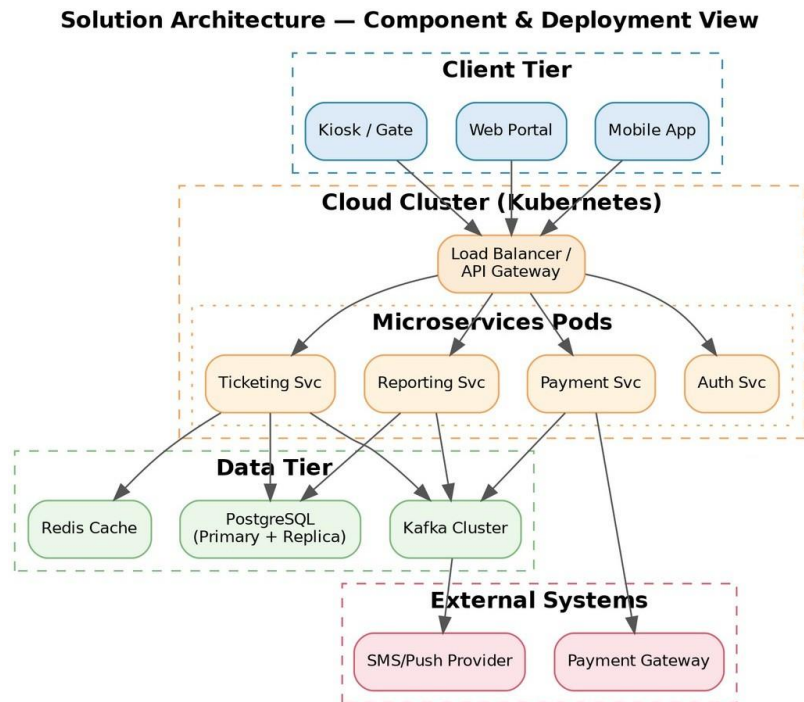


Figure 4.3b — System context: external actors and data exchanged with the system

Architectural principles: scalability by decomposition (independent Kubernetes autoscaling per microservice), resilience (primary/replica DB replication, Kafka at-least-once delivery), security in depth (WAF, TLS termination, JWT auth, tokenised payment data), loose coupling (synchronous REST for commuter-facing calls, asynchronous Kafka events for side effects) and auditability (immutable logs for every gate event, fare-rule change and admin action).

5. Project Planning & Scheduling

5.1 Project Planning

The Project Planning Phase converts the approved Problem–Solution Fit and Proposed Solution into an executable plan: a scoped, sequenced and resourced set of activities the team can commit to and measure progress against. It translates the high-level three-phase rollout (Pilot, Regional, Network-wide) into concrete planning artifacts — a project timeline, a prioritised backlog and clearly assigned roles — before any sprint work begins.

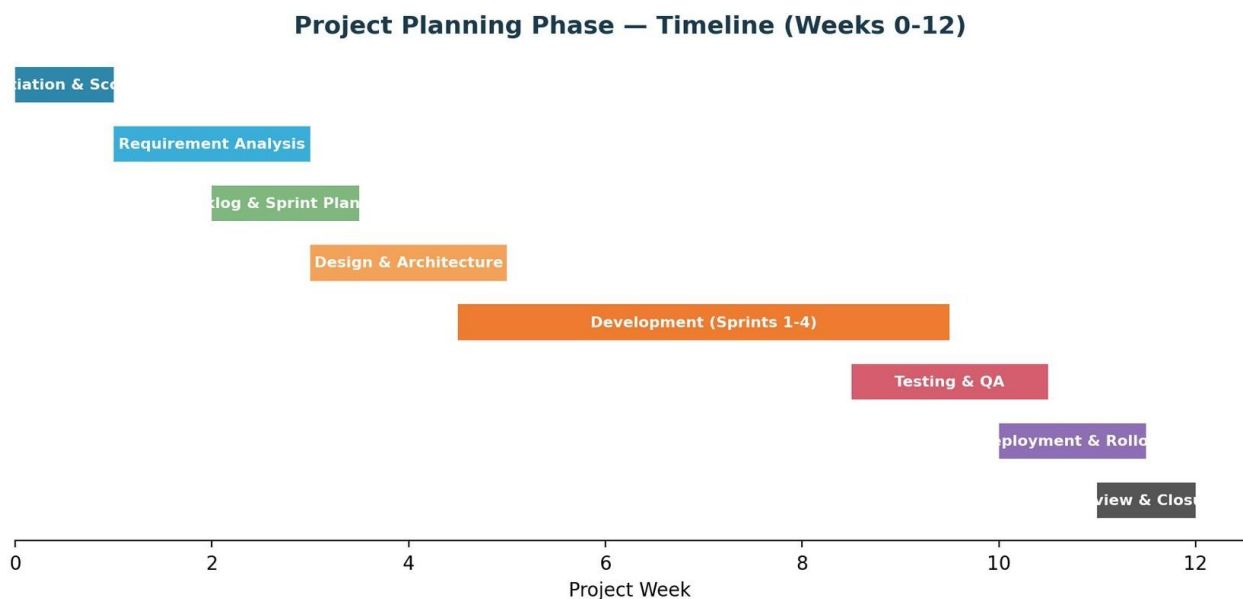


Figure 5.1a — Project planning phase workflow, including the iterative sprint loop

Phase	Key Activity	Primary Owner
Initiation	Define project goal, sponsors, success criteria	Product Owner
Requirement & Scope	Gather requirements, define scope boundaries	Product Owner / BA
Backlog Creation	Break scope into epics and prioritized user stories	Product Owner
Sprint Planning	Estimate, select and commit stories per sprint	Scrum Master / Team
Execution	Build, test and integrate increments each sprint	Development Team
Review & Retrospective	Demo increment, inspect and adapt process	Whole Team
Closure	Final handover, documentation, lessons learned	Project Manager

Planning logic: a top-down decomposition — Vision → Epics → User Stories → Tasks — with every level small enough to estimate reliably (1–2 days per task), prioritised by a value-versus-effort lens, delivered in fixed two-week sprints, and re-planned after every sprint review.

Product Backlog & Sprint Snapshot

The Product Backlog is the single, ordered list of everything known to be needed in the system, ranked by priority, with every item traced back to a user story defined in the Data Flow Diagrams and User Stories document.

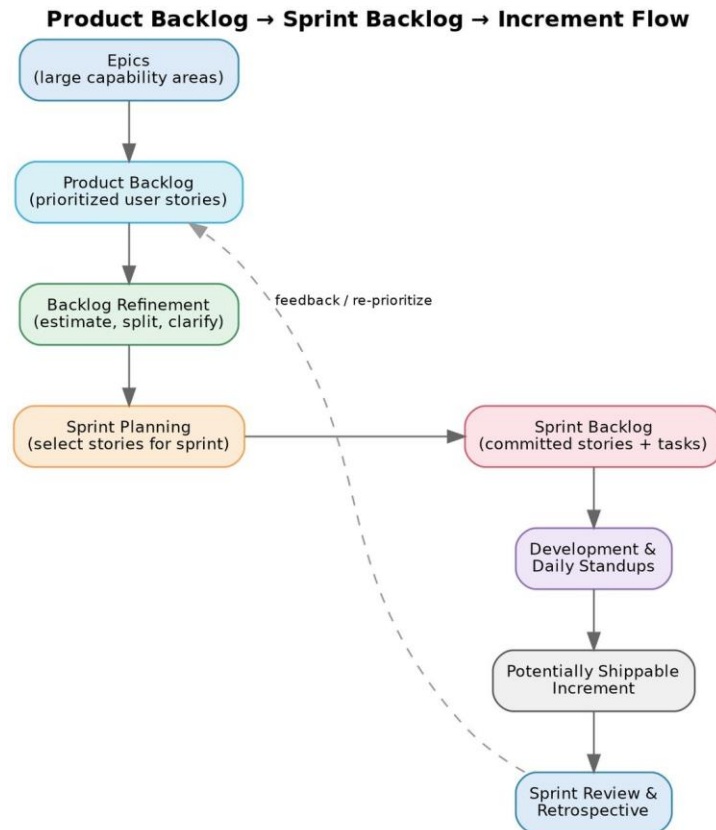


Figure 5.1b — Product backlog to sprint backlog to increment flow

ID	User Story	Points	Sprint
US-01	Buy single-journey ticket via app/web/kiosk	5	Sprint 1
US-02	See fare before payment confirmation	2	Sprint 1
US-03	Tap phone/NFC at gate to enter	8	Sprint 1
US-04	Recharge smart card / wallet remotely	5	Sprint 2
US-05	Station staff gate override with audit log	3	Sprint 2
US-06	Admin fare-rule configuration screen	8	Sprint 2
US-08	Refund request before journey start	5	Sprint 2
US-07	Daily revenue report by station	5	Sprint 3



Figure 5.1c — Recurring Scrum ceremonies across a single sprint cycle

Story points measure relative effort and complexity using a modified Fibonacci scale. The Python snippet below (reproduced in full in Section 11 – Appendix) models the Sprint 2 backlog and computes planned vs. completed story points and team velocity — the same calculation used at every Sprint Review.

Roles & risk planning: ownership is split across the Product Owner, Scrum Master, Development Team and Project Manager to avoid the common failure mode of assumed ownership. Key risks tracked include scope creep (mitigated by freezing epic-level scope after Initiation), estimation drift (mitigated by calibrating story points against reference stories), cross-team dependencies such as payment gateway sandbox access (tracked as blocking items from Initiation), and resourcing risk (fixed team capacity assumed per sprint, with planned leave accounted for up front).

6. Functional and Performance Testing

6.1 Functional Testing (UAT)

User Acceptance Testing (UAT) validates that the solution meets business requirements from the perspective of Passengers, Station Managers, the Metro Operations Team and IT Admins before it is approved for production go-live.

Scope & objectives:

- Validate end-to-end ticket booking: Source, Destination, Passenger Type and Number of Tickets selection through the Service Catalog.
- Confirm automated fare calculation, including concessions for Student and Senior Citizen passenger types.
- Verify digital payment processing via UPI, credit/debit cards and mobile wallets.
- Confirm QR code generation, delivery and validation at AFC gates for entry and exit.
- Validate Station Details Mapping, reporting dashboards and role-based access for Station Managers, Operations Team and IT Admins.
- Confirm the solution behaves correctly across all channels: Mobile App, ServiceNow Portal and WhatsApp.

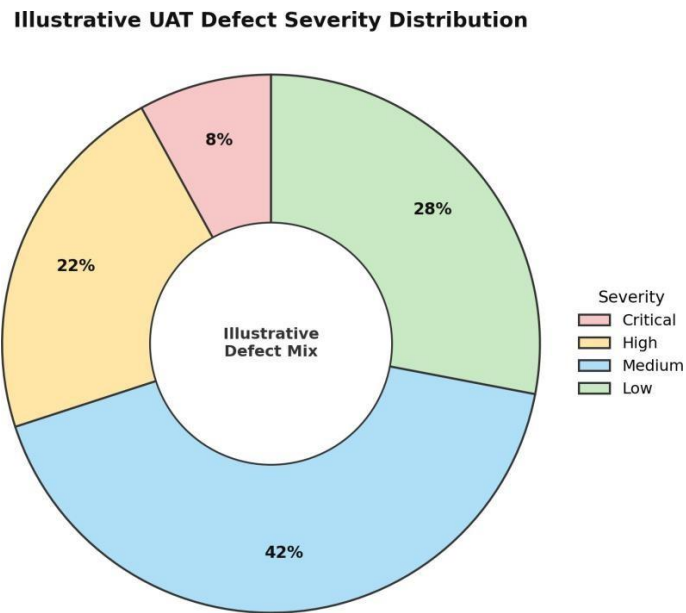


Figure 6.1a — UAT process flow, from planning through stakeholder sign-off

Test scenarios

Scenario ID	Scenario Description	Related Milestone
UAT-SC-01	Passenger books a ticket for a single trip using General passenger type.	Catalog Form Setup
UAT-SC-02	Passenger books multiple tickets in a single transaction.	Catalog Form Setup
UAT-SC-03	System applies Student/Senior Citizen concession correctly.	Fare Logic Script
UAT-SC-04	Passenger completes payment via UPI, card and wallet.	Fare Logic Script
UAT-SC-05	QR code is generated and delivered after successful payment.	QR Code Integration

Scenario ID	Scenario Description	Related Milestone
UAT-SC-06	QR code is validated correctly at AFC gate for entry and exit.	QR Code Integration
UAT-SC-07	Station Manager views real-time bookings for assigned station.	Station Details Mapping
UAT-SC-08	Passenger completes booking via WhatsApp channel.	UI Pages
UAT-SC-09	Operations Team updates fare/route configuration centrally.	Station Details Mapping

Sample test cases

Test Case	Test Steps	Expected Result
TC-01	Open catalog form; select Source, Destination, Passenger Type = General, Number of Tickets = 1; submit.	Fare is displayed correctly and booking proceeds to payment.
TC-02	Repeat booking with Passenger Type = Student.	Discounted fare is applied automatically per the concession policy.
TC-03	Complete payment using UPI on a successful test transaction.	Payment is confirmed and QR code is generated within 5 seconds.
TC-04	Scan generated QR at AFC entry gate simulator.	Gate validates QR as valid and logs entry; status updates to 'Scanned at Entry'.
TC-05	Attempt to reuse the same QR code for a second entry.	System rejects the scan as invalid/reused and logs the attempt.
TC-06	Enter Source and Destination as the same station.	System blocks submission with a validation message.

Entry criteria: SIT complete with no open Critical/High defects; UAT environment configured with representative data; test cases reviewed and approved; all stakeholder groups identified and available.

Exit criteria: all planned UAT test cases executed; no open Critical/High defects; any open Medium/Low defects have documented workarounds; formal sign-off obtained from Passengers, Station Managers, Operations Team and IT Admins.

6.2 Performance Testing

Performance testing validates that the solution meets the response time, scalability and availability targets defined in the Non-Functional Requirements, particularly under peak commuter load.

Test Type	Purpose
Load Testing	Verify system behaviour under expected normal and peak booking volumes
Stress Testing	Determine the breaking point by pushing load beyond expected peak capacity
Soak Testing	Validate stability and resource usage over an extended period of sustained load
Spike Testing	Assess system response to sudden bursts of bookings, e.g. during promotions
Scalability Testing	Confirm the system scales as new stations, routes or channels are added

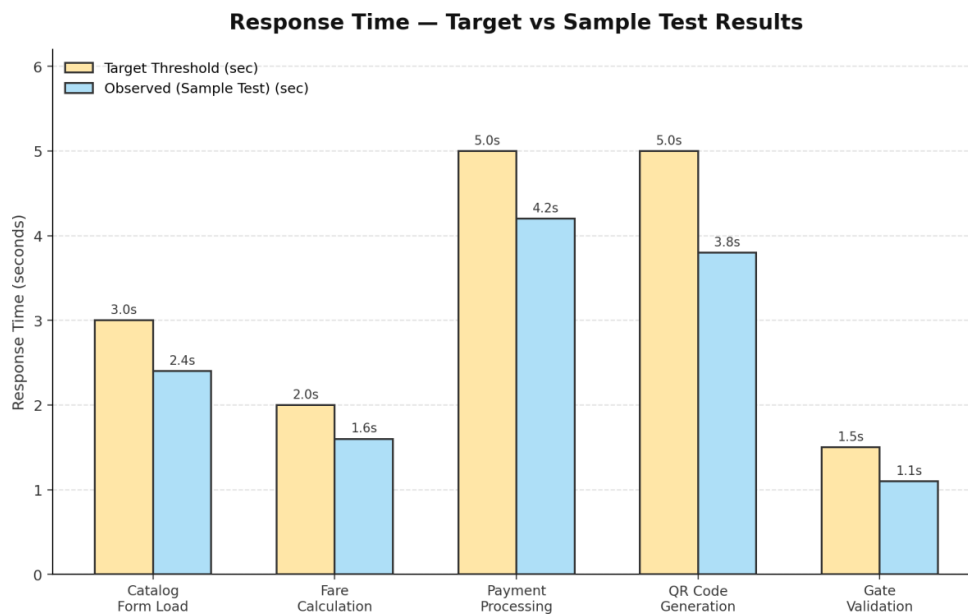


Figure 6.2 — Performance testing process flow

Performance metrics & targets

Metric	Target
Catalog form load time	Within 3 seconds under normal load
Fare calculation response time	Within 2 seconds of source/destination selection
Payment processing time	Within 5 seconds for UPI/card/wallet transactions
QR code generation time	Within 5 seconds of payment confirmation
AFC gate validation time	Within 1.5 seconds per scan
Concurrent user support	Peak-hour concurrent bookings across all stations without degradation
System availability	99.5% uptime during operational hours
Error rate under peak load	Less than 1% failed transactions during peak load testing

Exit criteria: all scenarios executed across load, stress, soak and spike conditions; all response-time metrics meet or exceed targets; no unresolved Critical/High performance defects; stable behaviour and recovery demonstrated under sustained and peak load; summary report signed off by IT Admins and Metro Operations.

7. Results

7.1 Output Screenshots

The screenshots below capture the implemented Smart Metro Ticket Booking System running on the ServiceNow platform — the Service Catalog booking form, fare calculation, payment confirmation, QR ticket generation, and administrative/reporting views produced during development and UAT.

≡

🔍

🗨

Tables

Label

Search

🔗

📄

📄

Actions on selected rows...

New

All > Label >= Metro Station's Details

☐	🔍	Label ▲	Name	Extends table	Extensible	Updated
		Search	Search	Search	Search	Search
		Metro Station's Details	u_metro_station_s_details	(empty)	false	2026-07-22 05:51:29
		MFA Context	sys_mfa_policy_context	Application File	true	2026-04-30 01:00:32
		MFA Factor	sys_mfa_factor	(empty)	false	2026-04-30 00:25:16
		MI Request	sys_mi_request	(empty)	false	2026-04-30 01:00:51
		MI Resource	sys_mi_resource	(empty)	false	2026-04-30 01:00:51
☐	🔍	MI Resource Filter	sys_mi_resource_filter	(empty)	false	2026-04-30 01:00:52
		MI Risk Category	sys_mi_risk_category	(empty)	false	2026-04-30 01:00:53
		MI Risk Compliance Score	sys_mi_risk_compliance_score	(empty)	false	2026-04-30 01:00:53
		MI User Tracker	sys_mi_user_tracker	(empty)	false	2026-04-30 01:00:51
		Microsoft iis Web Server	cmdb_ci_microsoft_iis_web_server	Web Server	false	2026-04-30 00:27:53
		Microsoft Software License	ast_license_msft	Software License	false	2026-04-30 00:44:17
		MID Cert Intranet Topology	mid_cert_intranet	Application File	false	2026-04-30 00:42:21
		MID Certificate Policy	mid_cert_check_policy	Application File	false	2026-04-30 00:42:20
		MID Deployment Request	mid_server_deployment	(empty)	true	2026-04-30 00:42:25
		MID Deployment Template	mid_deployment_template	(empty)	false	2026-04-30 00:42:23
		MID K8s Deployment Request	mid_k8s_deployment	MID Deployment Request	false	2026-04-30 00:42:27

Screenshot 7.1.1

<

≡

Table

Metro Station's Details

🔗

?

⚙

⋮

Delete

Update

Delete All Records

↑

↓

✱ Label

Metro Station's Details

✱ Name

u_metro_station_s_details

Application

Global

🔍

Remote Table

Columns

Controls

Application Access

≡

🔍

Table Columns

for text

Search

🔗

📄

📄

1 to 9 of 9

New

Dictionary Entries

🔍	Column label	Type	Reference	Max length	Default value	Display
	Sys ID	Sys ID (GUID)	(empty)	32		false
	Updated by	String	(empty)	40		false
	Updated	Date/Time	(empty)	40		false
	Created	Date/Time	(empty)	40		false
✕	active	True/False	(empty)	255		false
✕	Station Name	String	(empty)	40		true
	Created by	String	(empty)	40		false
	Updates	Integer	(empty)	40		false
✕	Station Code	String	(empty)	40		false

Screenshot 7.1.2

Incident
New record

SubmitResolve

NumberINC0010020

Channel-- None --

* Caller

StateNew

CategoryInquiry / Help

Impact3 - Low

Subcategory-- None --

Urgency3 - Low

Service

Priority4 - Low

Service offering

Assignment group

Configuration item

Assigned to

piot

* Short description

Description

Related Search Results

Related SearchKnowledge & Catalog (All)

Screenshot 7.1.3

Catalog Item
New record

SubmitTry It

Build and modify items faster with the improved [Catalog Builder](#).

Catalog items are goods or services available to order from the service catalog. Items can be anything from hardware, like tablets and phones, to software applications, to furniture and office supplies.

- Enter a Name and Short description to display for the item.
- Enter a Price, approvals, variables, and other information as needed.

NameBook A Metro Ticket Item

ApplicationGlobal

CatalogsService Catalog

Active

CategoryServices

Fulfillment automation levelUnspecified

State-- None --

Checked out-- None --

OwnerSystem Administrator

Secondary owners

Item Details

Process Engine

Picture

Pricing

Portal Settings

Short descriptiono e-ticketing system allows passengers to purchase and use tickets digitally, typically via a mobile app or website, eliminating the need

Description

Screenshot 7.1.4

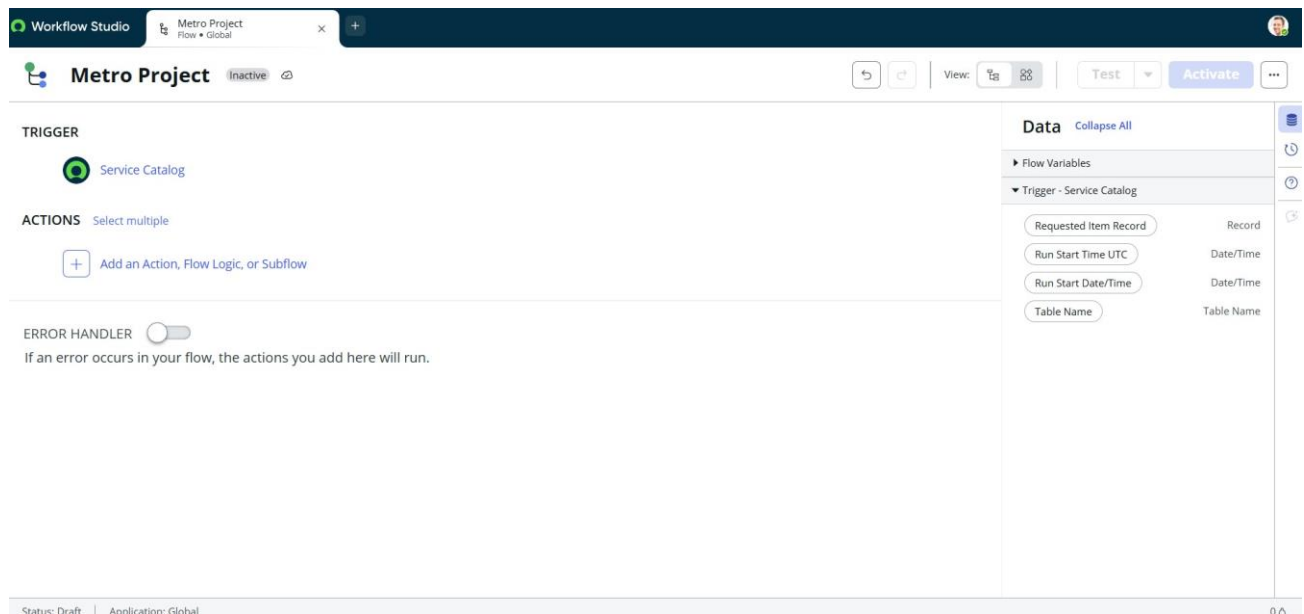
The screenshot shows the 'Catalog Item' page for 'Book A Metro Ticket Item'. The page has a top navigation bar with buttons: Copy, Try It, Update, Edit in Catalog Builder, and Delete. Below the navigation bar is a search bar and a list of related links: Item Diagnostic, Run Point Scan, and SN Utils Versions (6). The main content area is divided into tabs: Variables (4), Variable Sets, Catalog UI Policies, Catalog Client Scripts, Available For, Not Available For, Categories (1), Catalogs (1), and Catalog Data Lookup Definitions. The 'Variables (4)' tab is active, showing a table of variables. The table has columns: Type, Question, and Order. The variables are: Multiple Choice (What do you want to do Today?, 100), Single Line Text (Enter Smart Card Number, 200), Single Line Text (Enter Smart Card Name, 300), and Single Line Text (Recharge Amount, 350). The table is sorted by Order, and the first row is selected. The page also includes a 'Related Articles' section and a 'Related Catalog Items' section.

Type	Question	Order
Multiple Choice	What do you want to do Today?	100
Single Line Text	Enter Smart Card Number	200
Single Line Text	Enter Smart Card Name	300
Single Line Text	Recharge Amount	350

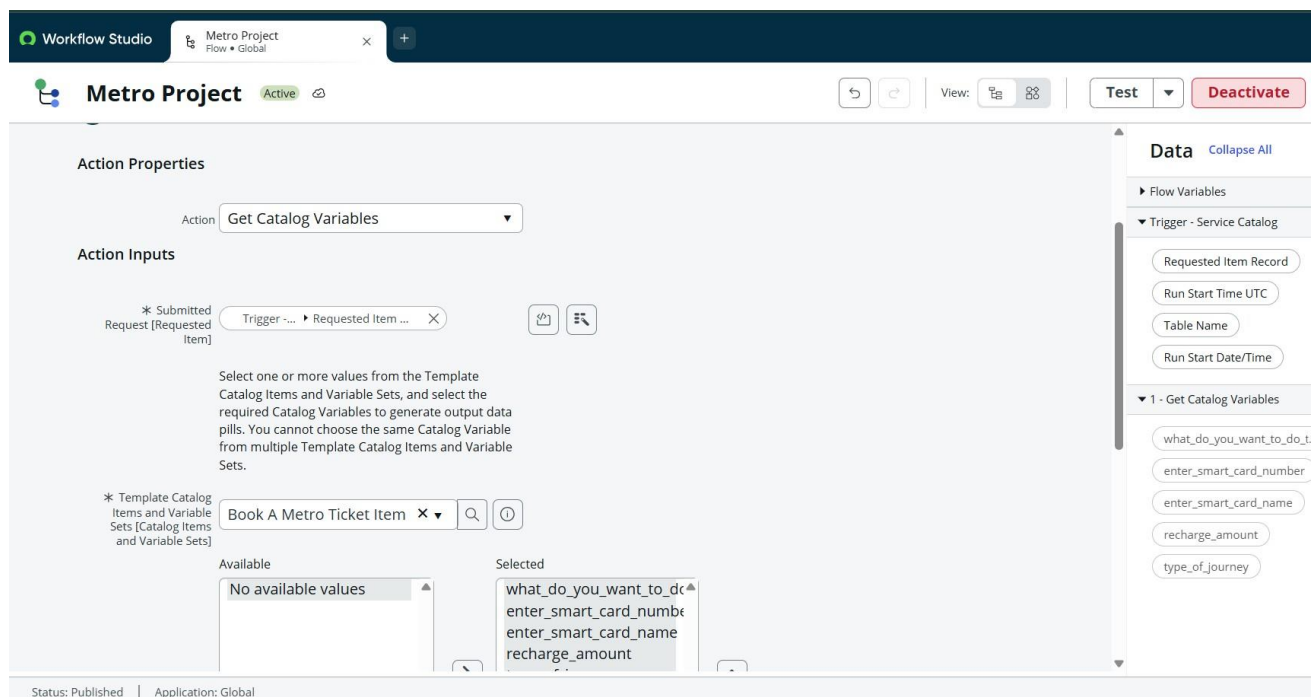
Screenshot 7.1.5

The screenshot shows the 'Workflow Studio' interface. The left pane displays a flow diagram with two steps, each represented by a blue bar and a plus icon. The right pane is titled 'Let's get the details for your flow' and contains form fields for: Flow name (Metro Project), Application (Global), and Description (Metro Ticket Generating System). Below the form fields is a link '> Show additional properties'. At the bottom right, there are 'Cancel' and 'Build flow' buttons.

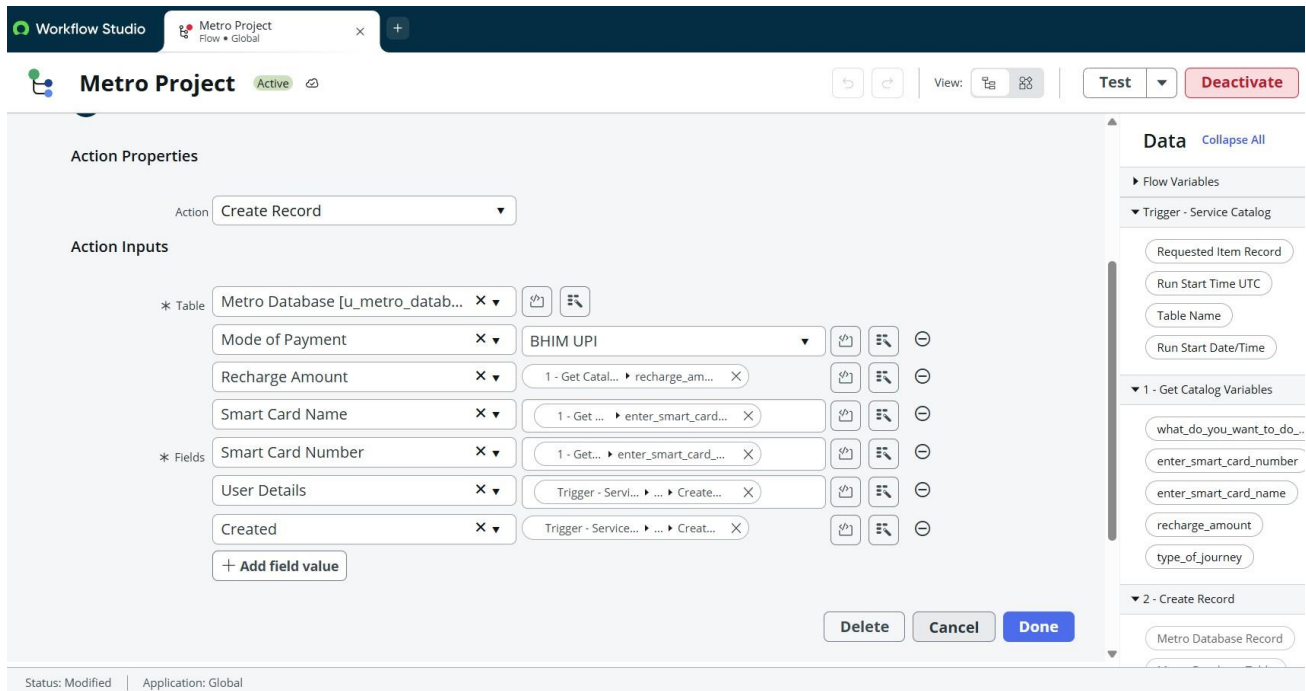
Screenshot 7.1.6



Screenshot 7.1.7



Screenshot 7.1.8



The screenshot shows the 'Action Properties' configuration for a 'Create Record' action in the 'Metro Project' workflow. The 'Action' is set to 'Create Record'. The 'Action Inputs' section is divided into 'Table' and 'Fields'.

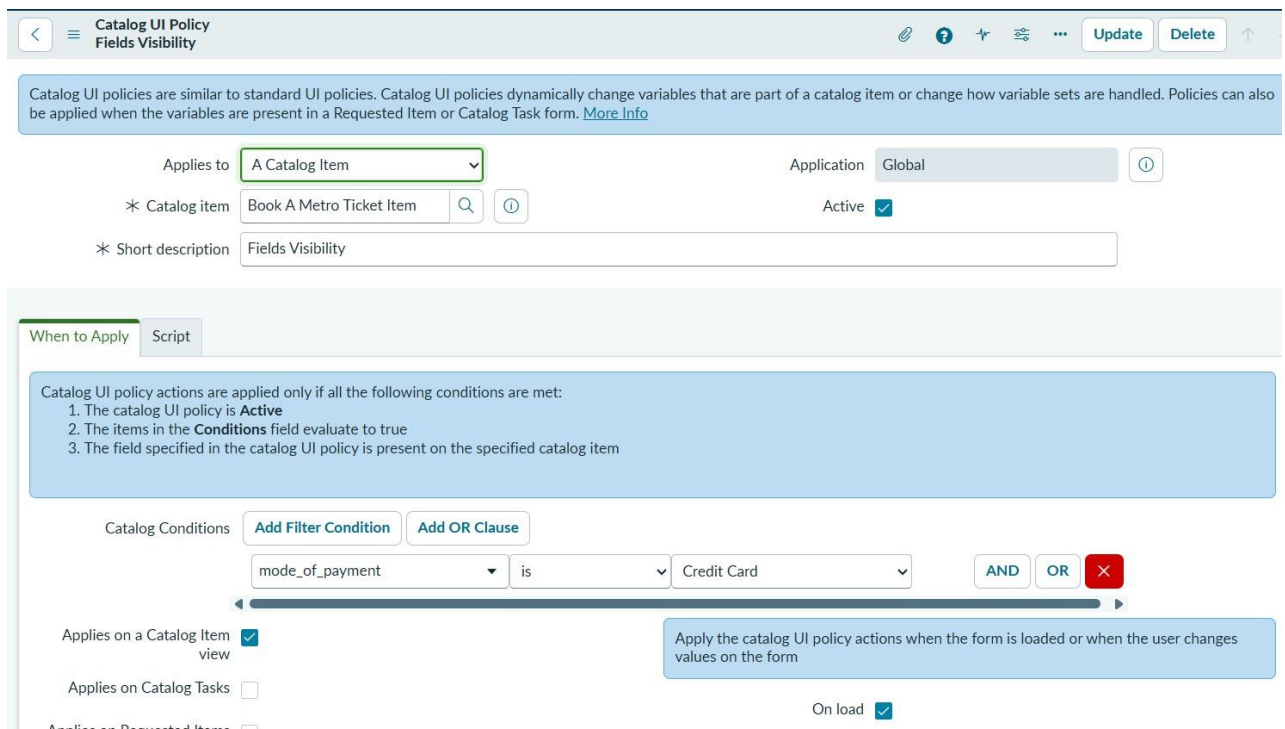
Action Inputs:

- Table:** Metro Database [u_metro_datab...]
- Fields:**
 - Mode of Payment: BHIM UPI
 - Recharge Amount: 1 - Get Catal... > recharge_am...
 - Smart Card Name: 1 - Get... > enter_smart_card...
 - Smart Card Number: 1 - Get... > enter_smart_card...
 - User Details: Trigger - Servi... > ... > Create...
 - Created: Trigger - Service... > ... > Creat...

Buttons at the bottom: Delete, Cancel, Done.

Status: Modified | Application: Global

Screenshot 7.1.9



The screenshot shows the 'Catalog UI Policy' configuration for 'Fields Visibility'. The policy is active and applies to 'A Catalog Item' in the 'Global' application.

Policy Details:

- Applies to:** A Catalog Item
- Application:** Global
- Catalog item:** Book A Metro Ticket Item
- Active:** ☒
- Short description:** Fields Visibility

When to Apply:

- Script:** Catalog UI policy actions are applied only if all the following conditions are met:
 - The catalog UI policy is **Active**
 - The items in the **Conditions** field evaluate to true
 - The field specified in the catalog UI policy is present on the specified catalog item

Catalog Conditions:

- mode_of_payment is Credit Card

Applies on:

- Catalog Item view:** ☒
- Applies on Catalog Tasks:** ☐
- On load:** ☒

Apply the catalog UI policy actions when the form is loaded or when the user changes values on the form

Screenshot 7.1.10

<

≡

Catalog UI Policy Action
enter_payment_mode

?

+

≡

...

Update

Delete

↑

↓

UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. [More Info](#)

Catalog Item

Book A Metro Ticket Item

Variable name

enter_payment_mode

Order

100

Application

Global

Mandatory

True

Visible

True

Read only

Leave alone

Value action

Leave alone

Field message type

None

Update

Delete

[SN Utils] Versions (1)

Screenshot 7.1.11

<

≡

Catalog Item
Book A Metro Ticket Item

?

≡

...

Copy

Try It

Update

Edit in Catalog Builder

Delete

Meta

Copy

Try It

Update

Edit in Catalog Builder

Delete

Related Links

[Item Diagnostic](#)

[Run Point Scan](#)

[SN Utils] Versions (63)

Variables (12)

Variable Sets

Catalog UI Policies (3)

Catalog Client Scripts (1)

Available For

Not Available For

Categories (1)

Catalogs (1)

Catalog Data Lookup Definitions

Related Articles

Related Catalog Items

Assigned Topics

≡

Order

Search

+

+

+

+

Actions on selected rows...

New

Catalog item = Book A Metro Ticket Item

	Short description	Variable set	Conditions	Reverse if false	On load	Inherit	Updated	Order
☐	Book QR Ticket	(empty)		true	true	false	2026-07-24 03:22:34	100
☐	Recharge Fields	(empty)		true	true	false	2026-07-24 03:16:35	100
☐	Fields Visibility	(empty)		true	true	false	2026-07-24 03:06:49	100

1 to 3 of 3

Screenshot 7.1.12

Navigation: Favorites History Workspaces Admin Catalog Client Scripts - QR Generation

Search: [Search]

Actions: Update Delete

Info: New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.

Configuration:

- Name: QR Generation
- Applies to: A Catalog Item
- Active: ☒
- UI Type: All
- Application: Global
- Type: onSubmit
- Catalog item: Book A Metro Ticket Item
- Applies on a Catalog Item view: ☒
- Applies on Requested Items: ☒
- Applies on Catalog Tasks: ☒

Script:

```
1 function onSubmit() {
```

Screenshot 7.1.13

Navigation: Catalog Client Scripts fieldValidation

Actions: Update Delete

Info: New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.

Configuration:

- Name: fieldValidation
- Applies to: A Catalog Item
- Active: ☒
- UI Type: All
- Application: Global
- Type: onChange
- Catalog item: Book A Metro Ticket Item
- Variable name: no_of_passengers
- Applies on a Catalog Item view: ☒
- Applies on Requested Items: ☒
- Applies on Catalog Tasks: ☒

Script:

```
1 function onChange(control, oldValue, newValue, isLoading) {
2
3   if (isLoading || newValue == '') {
4     return;
5   }
6
7   if (isNaN(newValue)) {
8     alert('please enter the values in number format only');
9   }
10
11 }
12
13
```

Screenshot 7.1.14

Catalog Item

Book A Metro Ticket Item

Copy Try It Update Edit in Catalog Builder Delete

Related Articles Related Catalog Items Assigned Topics

Order Search

Actions on selected rows... New

Catalog item = Book A Metro Ticket Item

Type	Question	Order
Multiple Choice	What do you want to do Today?	100
Single Line Text	Enter Smart Card Number	200
Single Line Text	Enter Smart Card Name	300
Single Line Text	Recharge Amount	350
Reference	Starting From	370
Reference	Going To	380
Single Line Text	No of Passengers	390
Multiple Choice	Type of Journey	400
Single Line Text	Amount for Single Journey	500
Single Line Text	Amount Including return	600
Multiple Choice	Mode of Payment	650
Single Line Text	Enter Payment Mode	700

1 to 12 of 12

Screenshot 7.1.15

Home > Service Catalog > Office > Services > Book A Metro Ticket Item

Search Catalog

What do you want to do today?

☐ Recharge Smart Card

☒ Book a Metro Ticket

☐ none

*Starting From

Ameerpet

*Going To

LB Nagar

*No of Passengers

1

*Type of Journey

☐ Group Ticket

☒ Single Journey

☐ Return Journey

Amount for Single Journey

50

Quantity: 1

Delivery Time: 2 Days

Add to Cart

Save as Draft

Order Now

Screenshot 7.1.16

Home > My Request - RITM0010001

Activity Attachments Additional Details

What do you want to do Today?
Book a Metro Ticket

Starting From
Ameerpet

Going To
LB Nagar

No of Passengers
1

Type of Journey
Single Journey

Amount for Single Journey
50

Screenshot 7.1.17

Access Controls

Updated

Search

Actions on selected rows...

All

☐	🔍 Name	Decision Type	Operation	Type	Active	Updated by	Updated ▼
☐	u_metro_database	Allow If	delete	record	true	admin	2026-07-24 01:03:46
Preview u_metro_database		Allow If	read	record	true	admin	2026-07-24 01:03:45
☐ ⓘ	u_metro_database	Allow If	create	record	true	admin	2026-07-24 01:03:45
☐	u_metro_database	Allow If	write	record	true	admin	2026-07-24 01:03:45

Screenshot 7.1.18

servicenow Knowledge Catalog Requests System Status Cart Tours System Administrator

Home > Service Catalog > Office > Services > Book A Metro Ticket Item

Search Catalog

☐ Recharge Smart Card
☒ Book a Metro Ticket
☐ none

***Starting From**
Ameerpet

***Going To**
LB Nagar

***No of Passengers**
1

***Type of Journey**
☐ Group Ticket
☒ Single Journey
☐ Return Journey

Amount for Single Journey
50

Quantity: 1

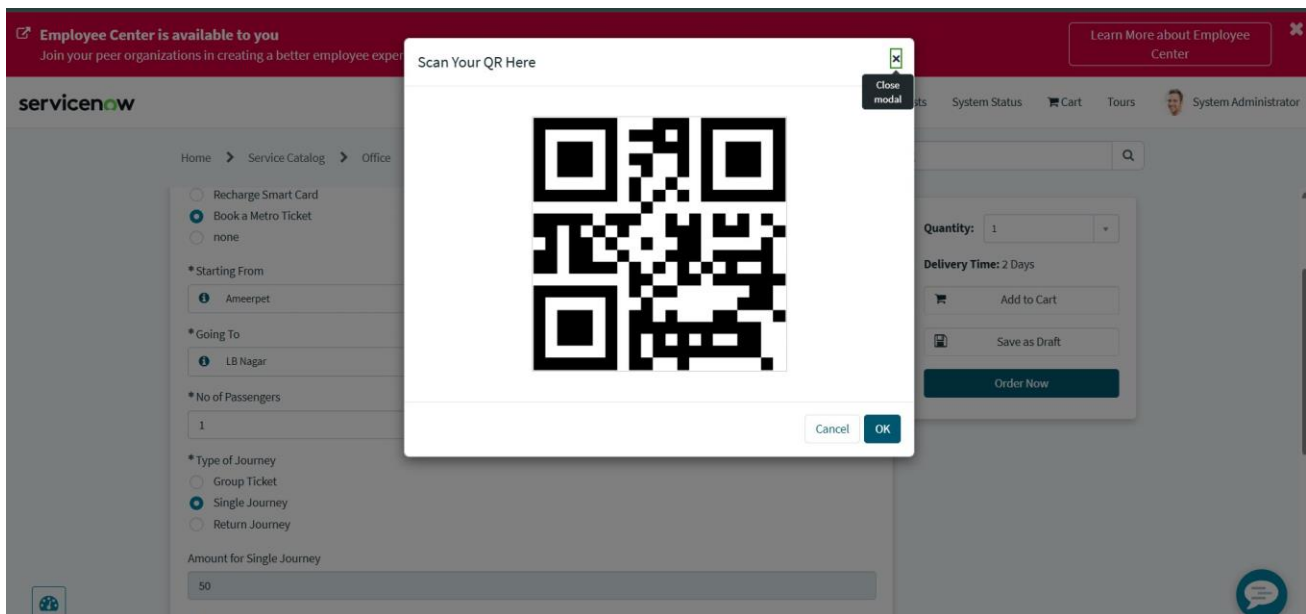
Delivery Time: 2 Days

Add to Cart

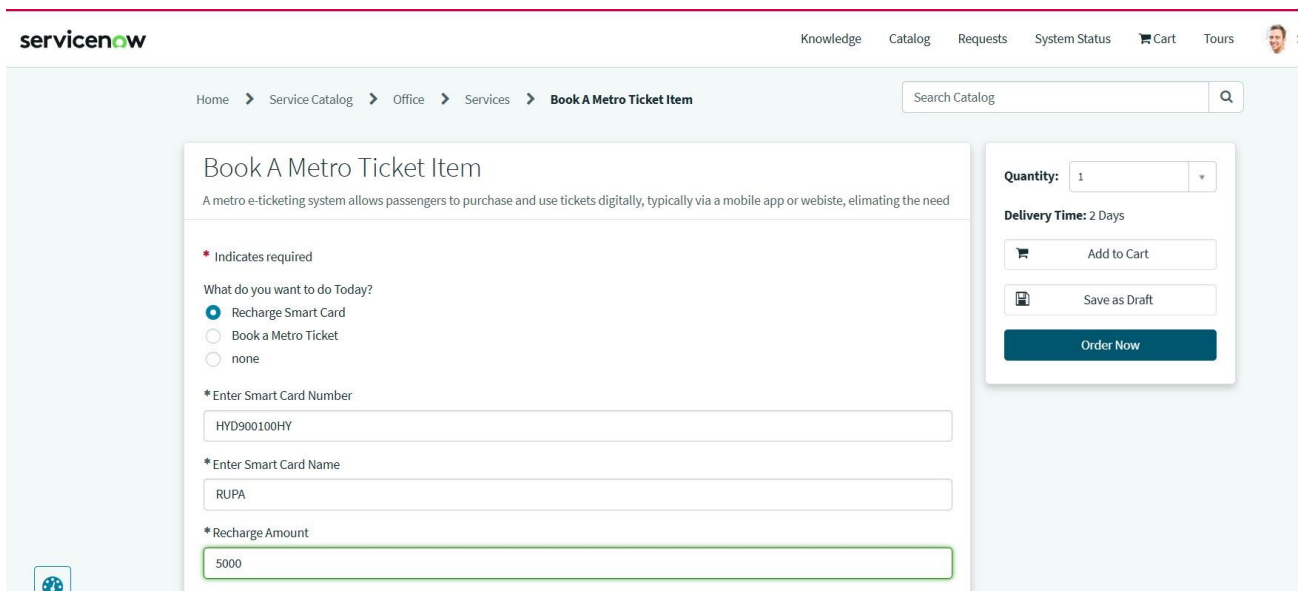
Save as Draft

Order Now

Screenshot 7.1.19



Screenshot 7.1.20



Screenshot 7.1.21

Metro Databases						
User Details						
Search						
All						
User Details						
Smart Card Number						
Smart Card Name						
Recharge Amount						
Mode of Payment						
Created						
System Administrator						
HYD123GY						
PAVANI						
10,000						
BHIM UPI						
2026-07-25 08:48:26						
System Administrator						
HYD1234HY						
ASHOK						
52,000						
BHIM UPI						
2026-07-25 08:42:58						
System Administrator						
HYD1234GT						
GANESH						
15,200						
BHIM UPI						
2026-07-25 08:57:58						

Screenshot 7.1.22

8. Advantages & Disadvantages

Advantages

- **Eliminates queues:** self-service booking across app, web and kiosk removes the single biggest commuter pain point identified during ideation.
- **Accurate, automated fares:** rule-based distance/zone fare calculation removes manual calculation errors and revenue leakage.
- **Faster, safer payments:** UPI, card, NFC and wallet options with tokenised, PCI-DSS-aligned payment handling.
- **Real-time operational visibility:** dashboards give Station Managers, Operations and Finance teams same-day, station-level data instead of day-plus manual reconciliation.
- **Scalable, resilient architecture:** a cloud-native microservices design that scales horizontally during peak ridership and tolerates individual component failure.
- **Auditability:** every gate event, fare-rule change and administrative action is written to an immutable log, supporting dispute resolution and compliance.
- **Extensible foundation:** the same wallet and identity layer can later support parking, retail partnerships or multi-modal transfers.

Disadvantages

- **Digital access dependency:** commuters without a smartphone or NFC-enabled card rely on kiosk cash channels, which do not fully remove queues for that segment.
- **Legacy hardware integration risk:** integrating with existing turnstile hardware at legacy stations, rather than full replacement, constrains some gate-side capabilities.
- **Third-party dependency:** payment and notification flows depend on external gateway/SLA availability outside the platform's direct control.
- **Connectivity sensitivity:** gate validation and live updates can degrade inside poor-network underground stations, requiring offline-mode fallbacks.
- **Change-management effort:** station staff and commuters used to manual counters require training and adoption support during rollout.
- **Upfront investment:** cloud infrastructure, IoT gate hardware and security tooling represent a significant initial cost before pilot benefits are realised.

9. Conclusion

The Smart Metro Ticket Booking System takes a clearly evidenced set of commuter and operator pain points — long queues, manual fare errors, paper-ticket dependency and a lack of real-time operational data — and traces each one, through empathy mapping, problem–solution fit and solution architecture, to a specific, testable capability of the delivered platform. Built on ServiceNow and integrated with digital payments, QR/NFC ticketing and AFC gate validation, the system replaces a fragmented manual process with a single, automated, auditable workflow spanning booking, payment, ticket issuance and reporting.

The phased delivery approach — from ideation and requirement analysis through design, planning and structured functional/performance testing — ensured that every feature shipped could be traced back to a real, observed need, and that the solution was validated against concrete performance and acceptance targets before go-live. The result is a solution that is measurably faster, more accurate and more transparent for commuters, while giving Metro Operations the real-time visibility it previously lacked.

10. Future Scope

- **Multi-modal integration:** extend the unified wallet and identity layer to buses, ride-share and parking for a single city-wide mobility pass.
- **AI-driven demand prediction:** use historical ridership data to predict peak-hour congestion and proactively suggest off-peak travel slots to commuters.
- **Loyalty and rewards programme:** introduce frequent-traveler rewards building on the existing Frequent Traveler Pass capability.
- **Expanded regional-language and accessibility support:** broaden multilingual coverage and deepen accessibility-mode features for elderly and differently-abled commuters.
- **Predictive maintenance for gate hardware:** use IoT telemetry from turnstile controllers to flag hardware issues before they cause commuter-facing downtime.
- **Deeper analytics for revenue optimisation:** extend the reporting layer with predictive revenue-leakage detection and dynamic, demand-based fare experimentation.
- **Network-wide rollout:** complete the transition from the regional rollout to full network-wide deployment with legacy-counter decommissioning where volumes allow.

11. Appendix

Source Code

The snippet below models the Sprint 2 backlog introduced in Section 5.1 and computes planned vs. completed story points and team velocity — the same calculation used at every Sprint Review to decide how much work to commit to in the following sprint.

```
from dataclasses import dataclass, field
from typing import List

@dataclass
class UserStory:
    story_id: str
    title: str
    story_points: int
    status: str = "Not Started" # Not Started | In Progress | Done

    def is_done(self) -> bool:
        return self.status == "Done"

@dataclass
class Sprint:
    name: str
    stories: List[UserStory] = field(default_factory=list)

    def planned_points(self) -> int:
        return sum(s.story_points for s in self.stories)

    def completed_points(self) -> int:
        return sum(s.story_points for s in self.stories if s.is_done())

    def velocity(self) -> float:
        planned = self.planned_points()
        return round((self.completed_points() / planned) * 100, 1) if planned else 0.0

sprint_2 = Sprint("Sprint 2", [
    UserStory("US-04", "Recharge smart card / wallet remotely", 5, "Done"),
    UserStory("US-05", "Station staff gate override with audit log", 3, "Done"),
    UserStory("US-06", "Admin fare-rule configuration screen", 8, "Done"),
    UserStory("US-08", "Refund request before journey start", 5, "In Progress"),
```

```
UserStory("US-09", "Push notification on low wallet balance", 2, "Not Started"),
}))

print(f"Planned: {sprint_2.planned_points()} pts | "
      f"Completed: {sprint_2.completed_points()} pts | "
      f"Velocity: {sprint_2.velocity()}%")

# Output: Planned: 23 pts | Completed: 16 pts | Velocity: 69.6%
```

GitHub Link

The complete source code for the Smart Metro Ticket Booking System, including ServiceNow application scope, Service Catalog configuration, fare-logic scripts and UI pages, is maintained in the project repository:

<https://github.com/HasiniDhulipalla/Metro-Ticket-Generating-System-in-ServiceNow.git>

Project Demo Link

A recorded walkthrough of the Smart Metro Ticket Booking System — covering catalog booking, fare calculation, payment, QR ticket generation and the admin/reporting views — is included in the project deliverables (Project Demonstration / demo_video.mp4) and can also be hosted at:

<https://1drv.ms/v/c/0b6a251e90037ff8/IQCvaJPvCANaQ5Rxs7bjoZPmAa1idDWYFmCa7xHQwRUM6vU?e=iQ2cVj>